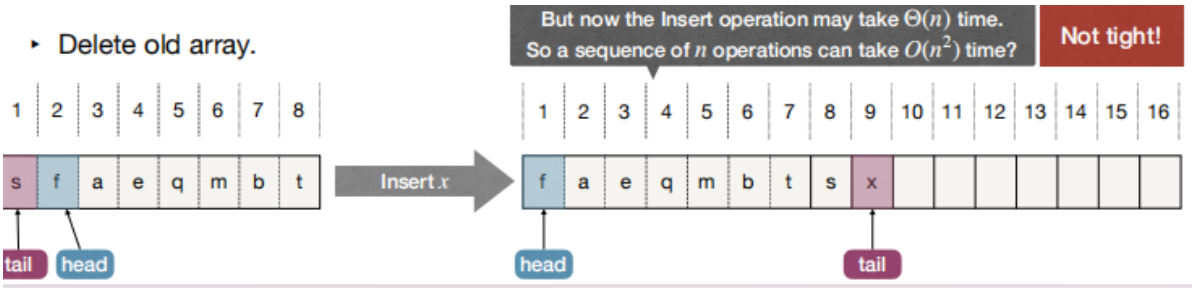


Amortized Analysis--平摊分析

1.Implement Queue with CircularArray

- 当数组满了时
 - 分配一个双倍大小的数组
 - 把已有的元素拷贝到新数组中，并插入新元素
 - 删除旧数组



“平均成本”的分析技巧:

- 通常用在数据结构的分析中
- 思想：即使必须执行昂贵的操作，通常也可以避免很少执行它们，这样每项操作的“平均”成本就不会那么高
- 注意：平摊分析不同于通常所说的平均案例分析，因为它没有对数据值的分布作出任何假设，而平均案例分析假设数据不是“坏的”。也就是说，平摊分析是一种最坏的情况分析，但是是针对一系列操作，而不是对于单个操作。

2.Aggregate method for Amortized Analysis--聚合法

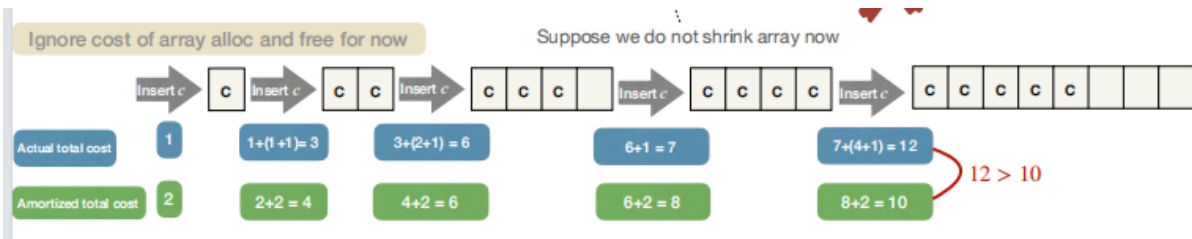
假设没有必要区别数据结构中的不同操作，则可以用聚合法

聚合法：把所有操作的成本相加然后除以操作的数量

3.Amortized Analysis

不同的操作也许会有不同的平摊成本

- 定义：对于每一个 $k \in N^+$, 操作有平摊成本 $\hat{c}(n)$, 任何 k 个操作的总成本满足 $\leq \sum_{i=1}^k \hat{c}(n_i)$, 其中 n_i 是进行第 i 次操作时数据结构的大小。
- 考虑一系列操作, c_i 为 i 次操作的总成本, $\hat{c}_i$ 为 i 次操作的平摊成本
- 对于任何 $k \in N^+$, 有 $\sum_{i=1}^k c_i \leq \sum_{i=1}^k \hat{c}_i$



4.平摊分析技巧

4.1 The Accounting Method--会计方法

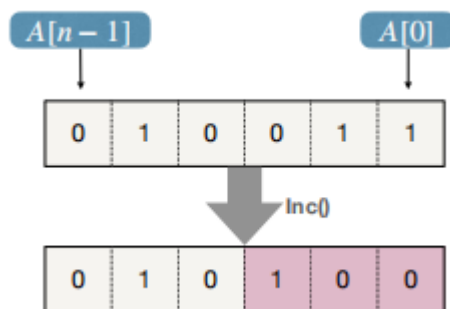
1. *Recall*: 考虑一系列操作, c_i 为 i 次操作的总成本, $\hat{c}_i$ 为 i 次操作的平摊成本。
对于任何 $k \in \mathbb{N}^+$, 有 $\sum_{i=1}^k c_i \leq \sum_{i=1}^k \hat{c}_i$ 。

- 想象你有一个银行账户 B , 初始余额为0
- 对于 i 次操作, 你花费了 $\hat{c}_i$ 的钱
 - 如果 $\hat{c}_i \geq c_i$, 向操作中支付 c_i 的钱, 并且把 $\hat{c}_i - c_i$ 的钱存进 B 中。
 - 如果 $\hat{c}_i < c_i$, 向操作中支付 c_i 的钱, 并且把 $c_i - \hat{c}_i$ 的钱从 B 中取出。
- 平摊分析是有效的当 $B = \sum_{i=1}^k (\hat{c}_i - c_i) > 0$ 时。

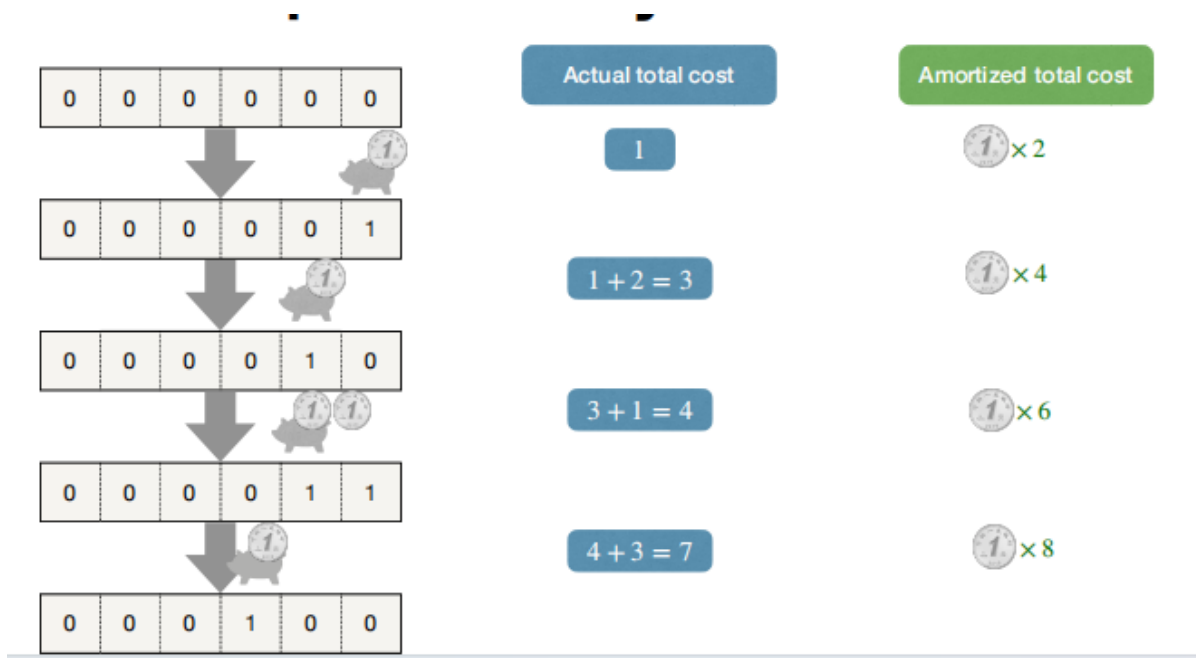
2. Example--Binary Counter(二进制计数器)

- 使用长度为 n 的二进制数组来代表一个数字
- 数字初始为0, 然后进行 *Inc* 操作, 把1加到这个数字上
- *Inc* 的成本: 翻转的位数
- *Inc* 的平均成本? $O(n)$

```
1 Inc(A):  
2   i = 0  
3   while i < n and A[i] = 1  
4       A[i] = 0  
5       i = i + 1  
6   if i < n  
7       A[i] = 1
```



- 在每一次 *Inc* 操作时: $0 \rightarrow 1$ 需要最多一位比特, 而 $1 \rightarrow 0$ 需要很多位比特。
- 如果我们进行 $0 \rightarrow 1$ 时存进去1, 而 $1 \rightarrow 0$ 则是免费的。
- 每次 *Inc* 操作最多做1次 $0 \rightarrow 1$ 操作, 因此平摊成本为 $2 = O(1)$ 。



4.2 The Potential Method--潜在方法

- 考虑一系列操作: c_i 是 i 次操作的总成本, $\hat{c}_i$ 是 i 次操作的平摊成本, 对于任何 $k \in N^+$, 有 $\sum_{i=1}^k c_i \leq \sum_{i=1}^k \hat{c}_i$.
- 设计一个势函数 Φ 把数据结构的状态映射到真实的值上去
- $\Phi(D_0)$: 数据结构的初始势, 通常设置为 0
- $\Phi(D_i)$: 在 i 次操作后数据结构的势
- 定义 $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$, 则 $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$
- 对于有效的平摊分析, 需要满足对于所有的 k , 有 $\Phi(D_k) \geq \Phi(D_0)$

Example: Binary Counter

- 如何定义 $\Phi(D_i)$?
- $\Phi(D_i)$ = 在第 i 次 **Inc** 操作后数组中 1 的数量
- $c_i = (\text{number of bits } 0 \rightarrow 1) + (\text{number of bits } 1 \rightarrow 0)$
- $\Phi(D_i) - \Phi(D_{i-1}) = (\text{number of bits } 0 \rightarrow 1) - (\text{number of bits } 1 \rightarrow 0)$
- $\hat{c}_i = 2 \cdot (\text{number of bits } 0 \rightarrow 1) \leq 2$

Back to CircularArray based Queue

- Now suppose we need to shrink array for space consideration
 - **Solution(1):** Reduce array size to half when array only half loaded after Remove. (Allocate new array of half size, copy items to new array, and delete old array.)
 - **Solution(2):** Reduce array size to half when array only 1/4 loaded after Remove. (Allocate new array of half size, copy items to new array, and delete old array.)
- Quiz: which one is better with respect to amortized cost?